

Object Oriented Software Engineering

Chapter Six

Mapping Models to Code and Testing

Mapping Models to Code

- If the **design pattern selection** and the specification of **class interfaces** were done carefully, **most design** issues should now be resolved.
- We could implement a system that realizes the **use cases specified during requirements elicitation and system design**.
- However, as developers start putting together the **individual subsystems developed** in this way, they are confronted with many integration problems.
- Different developers have probably handled contract violations differently.
- Undocumented parameters may have been added to the API to address a requirement change.

An Overview of Mapping

- A **transformation** aims at **improving one aspect of the model** (e.g., its modularity) while preserving all of its other properties (e.g., its functionality).
- These transformations occur during numerous **object design and implementation activities**.
- A transformation may **add, remove, or rename classes, operations, associations, or attributes**.
- A transformation can also **add information to the model or remove information from**
- We used transformations to **eliminate redundancy** from the analysis, design and implementation model.
- Developers transform the **associations of the object model** into collections of object references, because programming languages **do not support the concept** of association.

An Overview of Mapping

- Many programming languages also **do not** support contracts, so developers need to transform the contract specification of operations into code for detecting and handling contract violations.
- Developers revise the interface specification of an object to accommodate **new requirements** from the client.
- These transformations **are not** as intellectually challenging as other activities in the development process, such as analysis or system design, but they have a repetitive and mechanical aspect and are made difficult by the scale of the work involved.

An Overview of Mapping

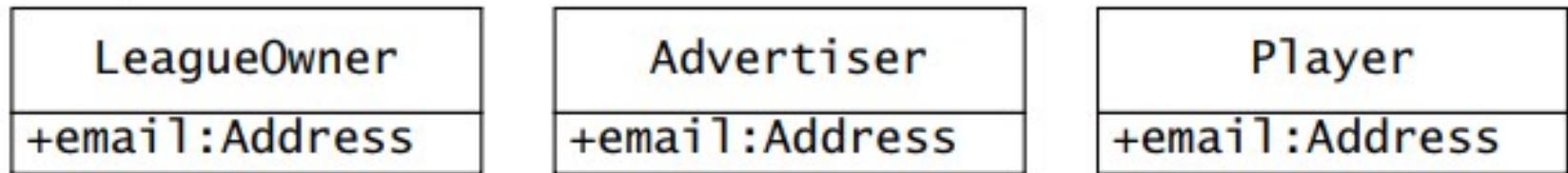
- There are Four Types of Transformation
 - **Model transformation** is operate on object models.
 - *Refactoring* are transformations that operate on source
 - *Forward engineering* produces a source code template that corresponds to an object model
 - *Reverse engineering* produces a model that corresponds to source code

An Overview of Mapping

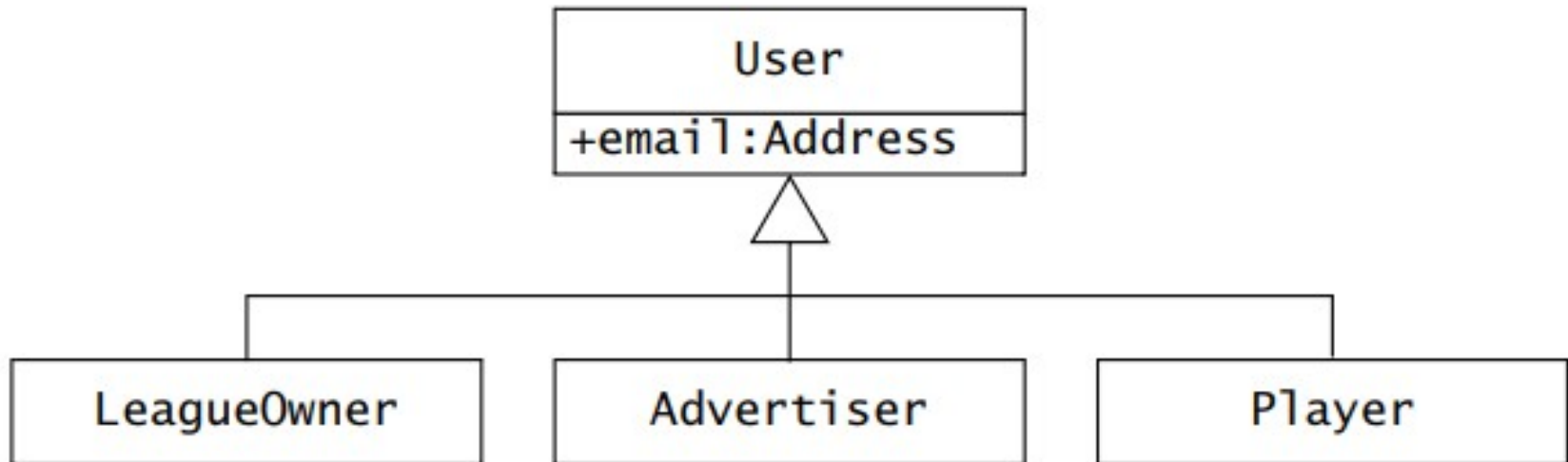
- **Model Transformation**
- A **model transformation** is applied to an object model and results in another **object model**.
- The purpose of object model transformation is to **simplify or optimize the original model**, bringing it into closer compliance with all requirements in the specification.
- An example is the conversion of a simple attribute (e.g., an address represented as a string) to a class (e.g. a class with street address, zip code, city, state, and country attributes).

An Overview of Mapping

Object design model before transformation



Object design model after transformation



Mapping Models to Code

- A **refactoring** is a transformation of the **source code** that improves its **readability or modifiability** without changing the behavior of the system

Before refactoring

```
public class Player {  
    private String email;  
    //...  
}  
  
public class LeagueOwner {  
    private String eMail;  
    //...  
}  
  
public class Advertiser {  
    private String email_address;  
    //...  
}
```

After refactoring

```
public class User {  
    protected String email;  
}  
  
public class Player extends User {  
    //...  
}  
  
public class LeagueOwner extends User  
{  
    //...  
}  
  
public class Advertiser extends User {  
    //...  
}
```

Mapping Models to Code

- **Forward engineering** is applied to a set of model elements and results in a set of corresponding source code statements, such as a class declaration, a Java expression, or a database schema.
- The purpose of forward engineering is to maintain a strong correspondence between the **object design model and the code**, and to reduce the number of errors introduced during implementation, thereby decreasing implementation effort.

Mapping Models to Code

Forward engineering

Object design model before transformation



Source code after transformation

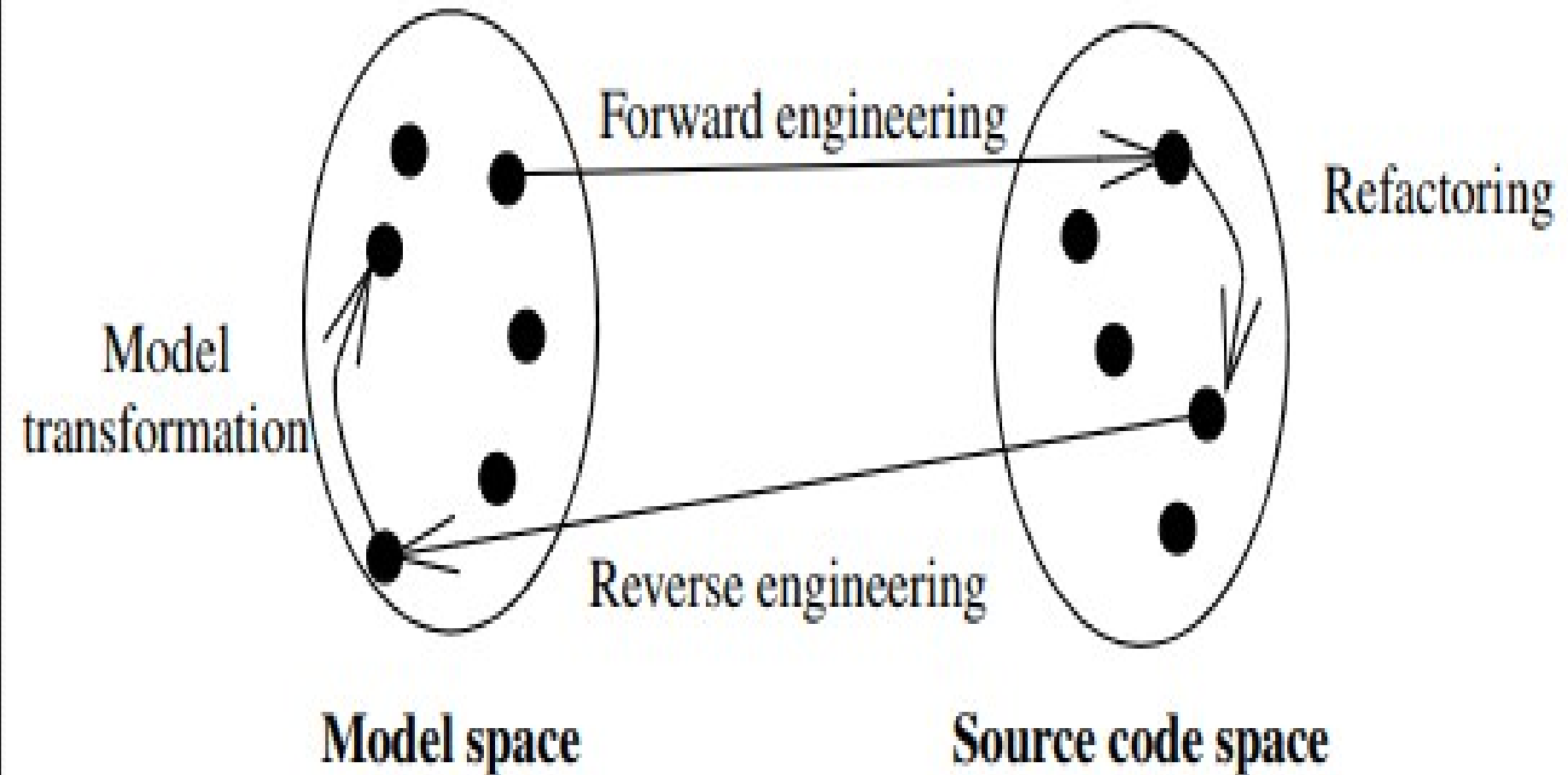
```
public class User {
    private String email;
    public String getEmail() {
        return email;
    }
    public void setEmail(String
value){
        email = value;
    }
    public void notify(String msg) {
        // ....
    }
    /* Other methods omitted */
}
```

```
public class LeagueOwner extends User
{
    private int maxNumLeagues;
    public int getMaxNumLeagues() {
        return maxNumLeagues;
    }
    public void setMaxNumLeagues
        (int value) {
        maxNumLeagues = value;
    }
    /* Other methods omitted */
}
```


Mapping Models to Code

- **Reverse engineering** is applied to a set of source code elements and results in a set of model elements. The purpose of this type of transformation is to recreate the model for an existing system, either because the model was lost or never created, or because it became out of sync with the source code. Reverse engineering is essentially an inverse transformation of forward engineering. Reverse engineering creates a UML class for each class declaration statement, adds an attribute for each field, and adds an operation for each method. However, because forward engineering can lose information (e.g., associations are turned into collections of references),

An Overview of Mapping



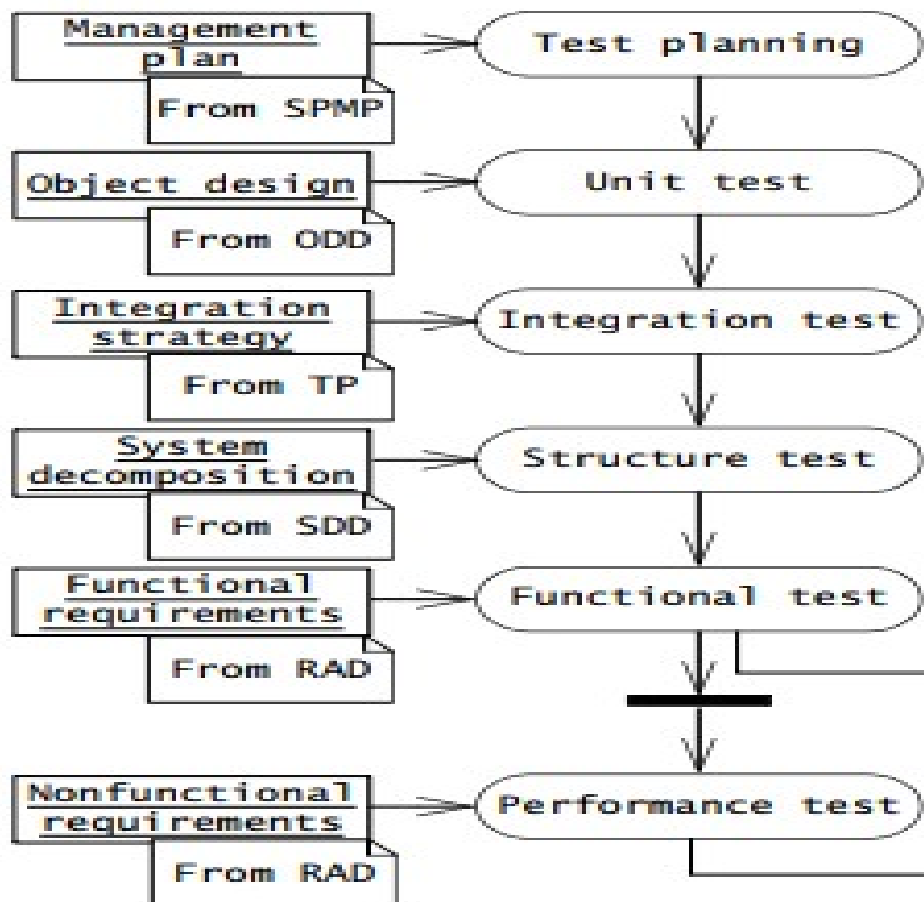
An Overview of Testing

- **Testing** is the process of **finding** differences between the **expected behavior specified by system models** and **the observed behavior of the implemented system**.
- **Unit testing** finds differences between a specification of an **object and its realization** as a component.
- **Structural testing** finds differences between the **system design model** and **a subset of integrated subsystems**.
- **Functional testing** finds differences between the use case model and the system.
- **Performance testing** finds differences between **nonfunctional requirements and actual system performance**.

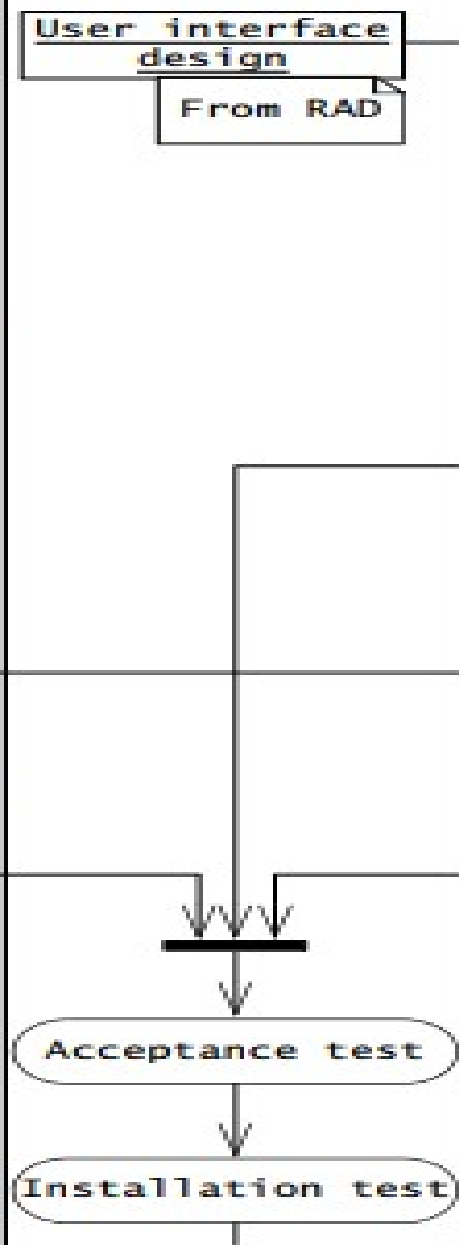
An Overview of Testing

- **Software reliability** is the probability that a software system **will not cause system failure** for a specified time under specified conditions.
- **Failure** is any deviation of the observed behavior from the specified behavior.
- A **fault**, also called “**defect**” or “**bug**,” is the **mechanical or algorithmic** cause of an erroneous state.
- The goal of testing is to **maximize the number of discovered faults**, which then allows **developers to correct them and increase the reliability of the system**.

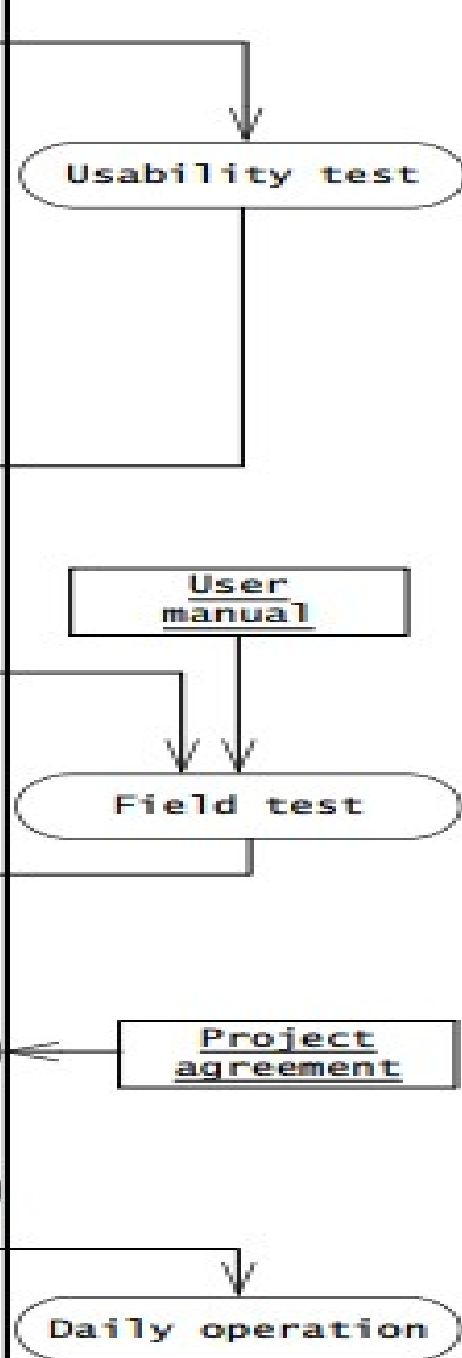
Developer



Client



User



An Overview of Testing

- A **fault**, also called **bug or defect**, is a design or coding mistake that may cause **abnormal component behavior**.
- An **erroneous state** is a manifestation of a fault during the execution of the system.
- An erroneous state is caused by one or more faults and can lead to a failure.
- A **failure** is a deviation between the specification and the actual behavior.
- A failure is triggered by one or more erroneous states. Not all erroneous states trigger a failure.
- A **test case** is a set of inputs and expected results that exercises a test component with the purpose of causing failures and detecting faults.
- A **test stub** is a partial implementation of components on which the tested component depends. A **test driver** is a partial implementation of a component that depends on the test component. Test stubs and drivers enable components to be isolated from the rest of the system for testing.

SOFTWARE QUALITY ASSURANCE

- *In the early history of computers, live bugs could be a problem (see Bugs and Debugging). Moths and other forms of natural insect life no longer trouble digital computers.*
- *However, bugs and the need to debug programs remain. In a 1966 article in Scientific American, computer scientist Christopher Strachey wrote,*
- *Although programming techniques have improved immensely since the early years, the process of finding and correcting errors in programing-"debugging" still remains a most difficult, confused and unsatisfactory operation.*
- *The chief impact of this state of affairs is psychological.*

SOFTWARE QUALITY ASSURANCE

- *Precisely speaking, the elimination of the syntactical bug is the process of debugging, whereas the detection and elimination of the logical bug is the process of testing.*
- *Gruenberger writes, The logical bugs can be extremely subtle and may need a great deal of effort to eliminate them. It is commonly accepted that all large software systems(operating or application) have bugs remaining in them.*

SOFTWARE QUALITY ASSURANCE

■ **QUALITY ASSURANCE TESTS**

- *One reason why quality assurance is needed is because computers are infamous for doing what you tell them to do, not necessarily what you want them to do. To close this gap, the code must be free of errors or bugs that cause unexpected results, a process called debugging..*
- *Scenario-based testing, also called usage-based testing, concentrates on what the user does, not what the product does. This means capturing use cases and the tasks users perform, then performing them and their variants as tests. These scenarios also can identify interaction bugs. They often are more complex and realistic than error-based tests.*
- *Scenario-based tests tend to exercise multiple subsystems in a single test, because that is what users do. The tests will not find everything, but they will cover at least the higher visibility system interaction bugs .*

TESTING STRATEGIES

- *The extent of testing a system is controlled by many factors, such as the risks involved, limitations on resources, and deadlines. In light of these issues, we must deploy a testing strategy that does the "best" job of finding defects in a product within the given constraints. There are many testing strategies, but most testing uses a combination of these: black box testing, white box testing, top-down testing, and bottom-up testing.*
- *However, no strategy or combination of strategies truly can prove the correctness of a system; it can establish only its "acceptability."*

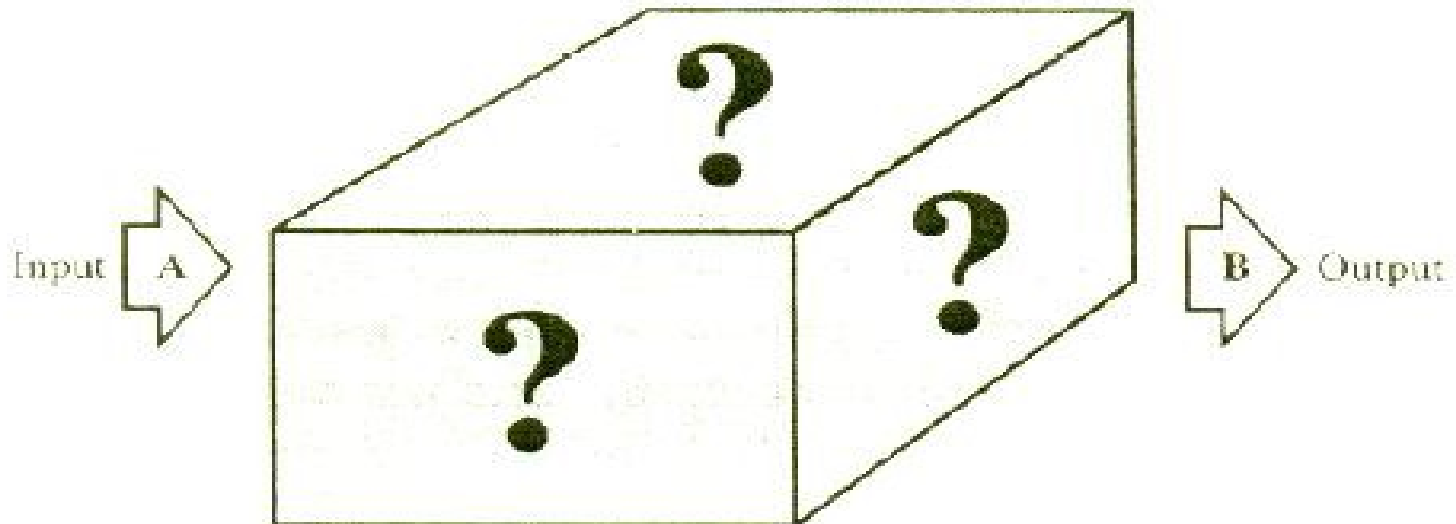
TESTING STRATEGIES

- **Black Box Testing**
- *The concept of the black box is used to represent a system whose inside workings are not available for inspection . In a black box, the test item is treated as "black," since its logic is unknown; all that is known is what goes in and what comes out, or the input and output.*
- *Weinberg describes writing a user manual as an example of a black box approach to requirements. The user manual does not show the internal logic, because the users of the system do not care about what is inside the system.*
- *In black box testing, you try various inputs and examine the resulting output; you can learn what the box does but nothing about how this conversion is implemented.*

TESTING STRATEGIES

- *Black box testing works very nicely in testing objects in an object-oriented environment.*
- *The black box testing technique also can be used for scenario-based tests, where the system's inside may not be available for inspection but the input and output are defined through use cases or other analysis information.*

The black box is an imaginary box that hides its internal workings.



TESTING STRATEGIES

- **White Box Testing**
- *White box testing assumes that the specific logic is important and must be tested to guarantee the system's proper functioning.*
- *The main use of the white box is in error based testing, when you already have tested all objects of an application and all external or public methods of an object that you believe to be of greater importance.*
- *In white box testing, you are looking for bugs that have a low probability of execution, have been carelessly implemented, or were overlooked previously .*

TESTING STRATEGIES

- *One form of white box testing, called path testing, makes certain that each path in a object's method is executed at least once during testing.*
- *Two types of path testing are statement testing coverage and branch testing coverage : .Statement testing coverage.*
- *The main idea of statement testing coverage is to test every statement in the object's method by executing it at least once.*
- *Murray states, "Testing less than this for new software is unconscionable and should be criminalized" [quoted in 2].*
- *However, realistically, it is impossible to test a program on every single input, so you never can be sure that a program will not fail on some input.*

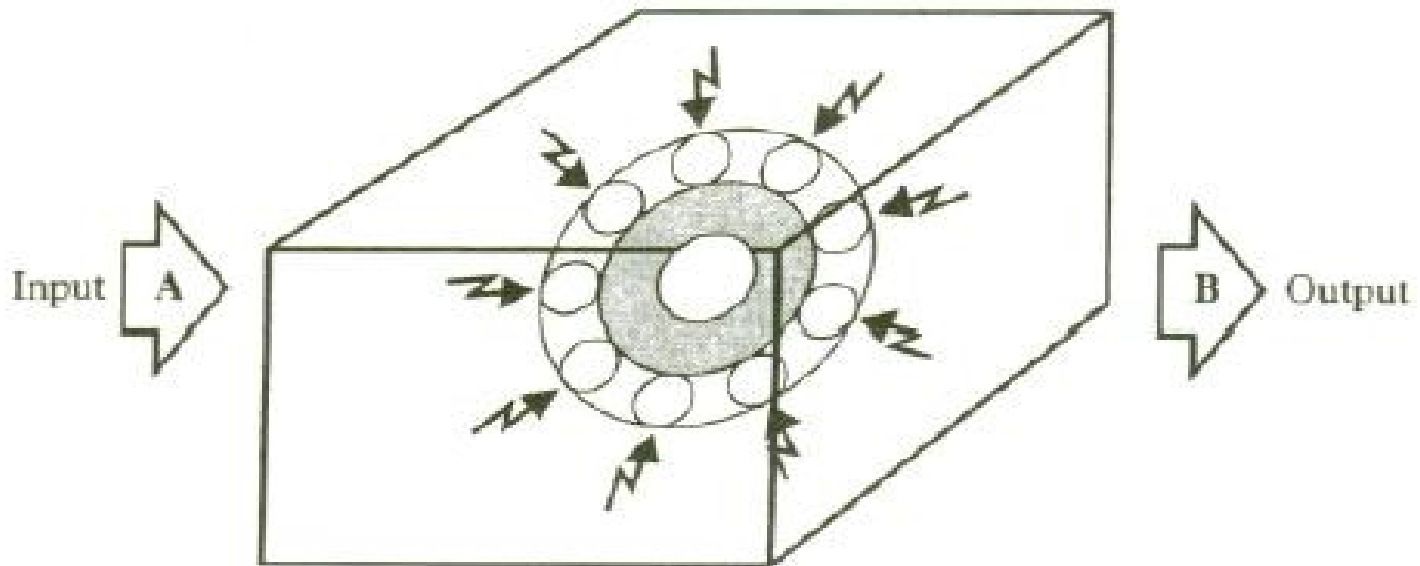
TESTING STRATEGIES

- ***Branch testing coverage.***
- *The main idea behind branch testing coverage is to perform enough tests to ensure that every branch alternative has been executed at least once under some test .*
- *As in statement testing coverage, it is unfeasible to fully test any program of considerable size.*
- *Most debugging tools are excellent in statement and branch testing coverage.*
- *White box testing is useful for error-based testing.*

TESTING STRATEGIES

- **Top-Down Testing**
- *Top-down testing assumes that the main logic or object interactions and systems messages of the application need more testing than an individual object's methods or supporting logic.*
- *A top-down strategy can detect the serious design flaws early*

In a white-box testing strategy, the internal workings are known.



TESTING STRATEGIES

- *In theory, top-down testing should find critical design errors early in the testing process and significantly improve the quality of the delivered software because of the iterative nature of the test.*
- *A top-down strategy supports testing the user interface and event-driven systems.*
- *Testing the user interface using a top-down approach means testing interface navigation. This serves two purposes, according to Conger.*
- *First, the top-down approach can test the navigation through screens and verify that it matches the requirements.*
- *Second, users can see, at an early stage, how the final application will look and feel .*
- *This approach also is useful for scenario-based testing.*
- *Top-down testing is useful to test subsystem and system integration.*

TESTING STRATEGIES

- **Bottom-Up Testing**
- *Bottom-up testing starts with the details of the system and proceeds to higher levels by a progressive aggregation of details until they collectively fit the requirements for the system.*
- *This approach is more appropriate for testing the individual objects in a system. Here, you test each object, then combine them and test their interaction and the messages passed among objects by utilizing the top-down approach.*
- *In bottom-up testing, you start with the methods and classes that call or rely on no others. You test them thoroughly. Then you progress to the next level up: those methods and classes that use only the bottom level ones already tested. Next, you test combinations of the bottom two layers. Proceed until you are testing the entire program.*

TESTING STRATEGIES

- *This strategy makes sense because you are checking the behavior of a piece of code before it is used by another.*
- *Bottom-up testing leads to integration testing, which leads to systems testing.*
- **Summary**
- *Testing may be conducted for different reasons.*
- *Quality assurance testing looks for potential problems in a proposed design.*
- *Guidelines and the basic concept of test plans, and for the most part, use cases can be used to describe the usage test cases.*
- *Some techniques, strategies, and approaches for quality assurance testing and the impact of object orientation on testing.*

Thank you for ur attention

Questions?